

SARCOOS

ESP32 ROBOTIC ECOSYSTEM

Technical Manual & API Reference

Revision 1.0 | Embedded Software & IoT
Framework: ESP-IDF (C++) | Platform: Espressif ESP32

PROJECT INFORMATION	
Developer	Ziad Mohamed Fathy
Specialization	Embedded Software & IoT — Full Software Stack (Solo)
University	Helwan International Technology University (HITU)
Supervisor	Dr. Ahmed Swedan
Academic Year	2024 / 2025

SUIT (Input Unit) Sensors & Potentiometers Button Inputs MQTT Publisher	ROVER (Output Unit) BTS7960 Motors Position Control MQTT Subscriber
--	--

Connected via WiFi / MQTT Broker

Table of Contents

1. System Architecture Overview

The Sarcoos ESP32 Robotic Ecosystem is a professional-grade, modular C++ framework designed for bidirectional robotic control using two ESP32 nodes. The Suit acts as the input terminal—gathering sensor and operator data—while the Rover acts as the output terminal, executing physical motion commands received over MQTT.

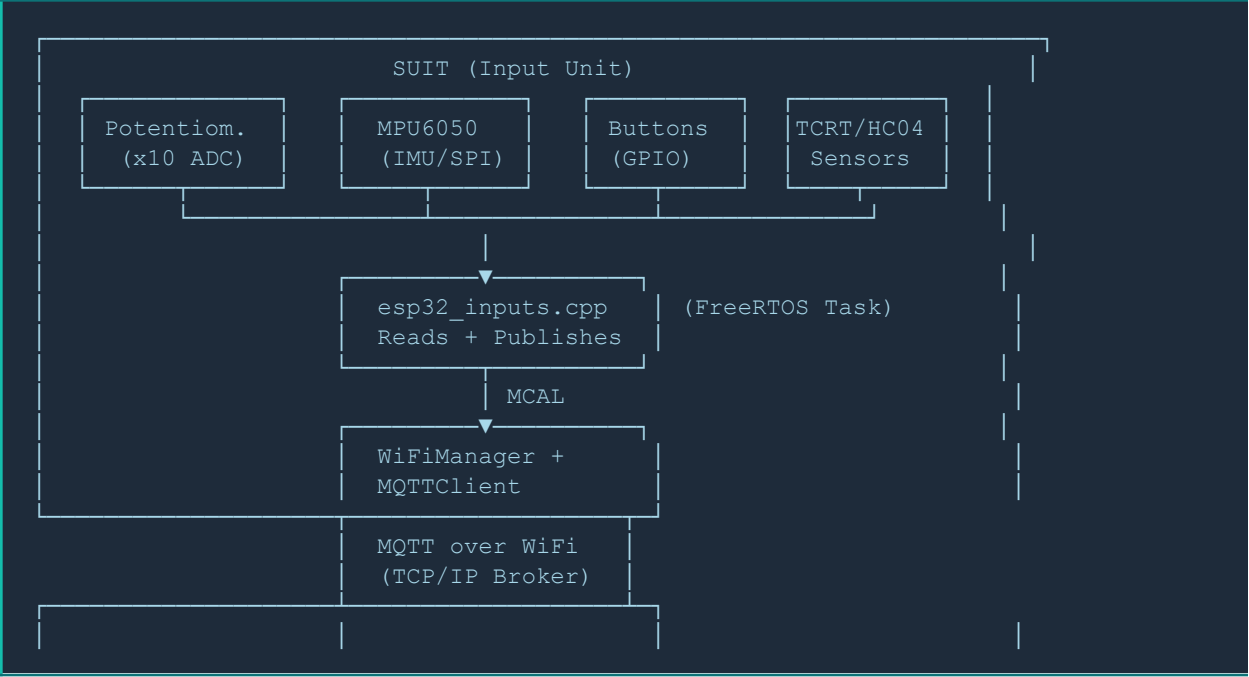
1.1 Tiered Architecture

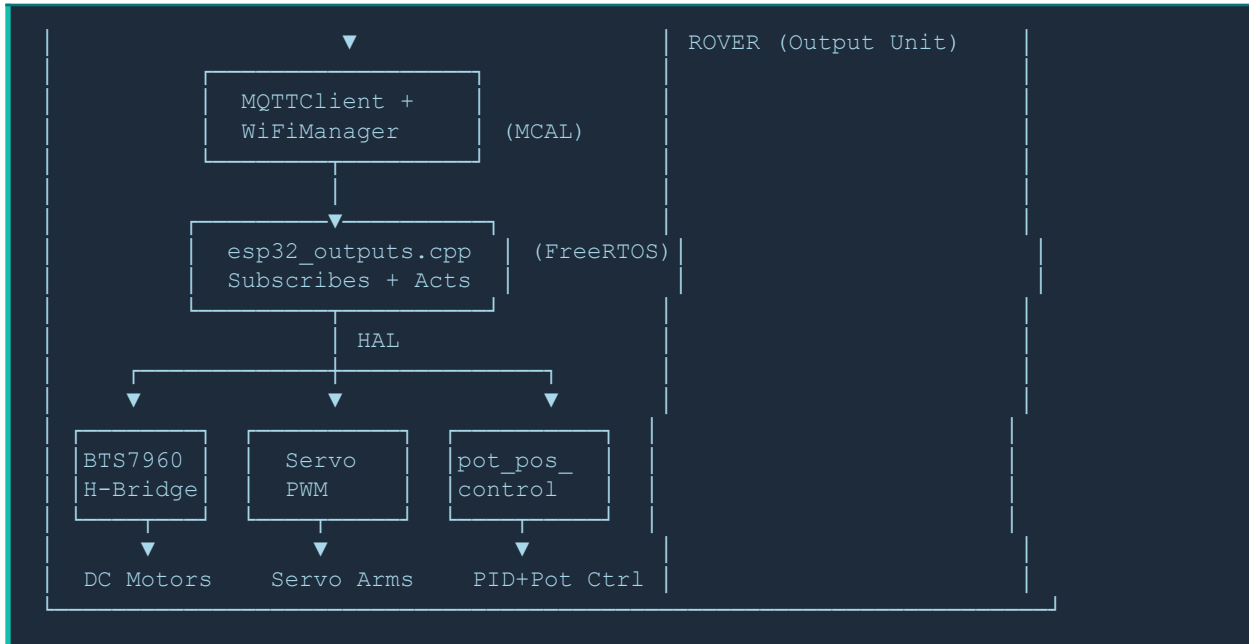
The framework is organized into three strict tiers, each with clearly defined responsibilities:

Tier	Layer	Responsibilities
Tier 3 (Top)	Application Layer	main.cpp, esp32_inputs.cpp, esp32_outputs.cpp, pot_pos_control.cpp — user-space logic, task orchestration, business rules
Tier 2 (Mid)	HAL — Hardware Abstraction Layer	BTS7960, Servo, MPU6050, Potentiometer, PID, Button, LED, Ultrasonic, TCRT5000, dc_motor — device drivers and peripheral interfaces
Tier 1 (Base)	MCAL — Microcontroller Abstraction Layer	WiFiManager, MQTTClient, NVS, UART — silicon-level abstractions for connectivity, storage, and communication

1.2 System Topology Diagram

The following ASCII diagram represents the full system topology from input capture to motor actuation:





1.3 Data Flow Summary

- Operator physically manipulates the Suit (moves potentiometers, presses buttons)
- esp32_inputs.cpp reads all hardware states via HAL drivers at a fixed FreeRTOS tick rate
- Readings are serialized and published to discrete MQTT topics (e.g., /sarcoos/pot/1)
- The MQTT broker routes messages to the Rover's MQTTClient subscriber
- esp32_outputs.cpp parses incoming values and dispatches commands to HAL motor drivers
- BTS7960 / BTS7960_Pot_Position execute closed-loop or open-loop motion

2. Hardware Abstraction Layer (HAL)

The HAL provides a unified C++ interface to all peripheral hardware. Each driver encapsulates pin configuration, communication protocols, and signal conditioning. Application code interacts exclusively through HAL APIs; raw register access and GPIO manipulation are confined to this layer.

HAL-01 **BTS7960 — High-Power H-Bridge Motor Driver**

2.1.1 Purpose & Hardware Context

The BTS7960 is a half-bridge IC rated for continuous currents up to 43A. Two ICs are paired per channel to form a full H-Bridge. The driver class abstracts direction and PWM control through the ESP32's LEDC peripheral.

2.1.2 Constructor

```
BTS7960(gpio_num_t r_en,    // Right enable pin (active HIGH)
        gpio_num_t l_en,    // Left  enable pin (active HIGH)
        gpio_num_t r_pwm,   // Right PWM signal (LEDC channel)
        gpio_num_t l_pwm,   // Left  PWM signal (LEDC channel)
        uint8_t    freq_hz = 20000); // PWM frequency (Hz)
```

2.1.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>BTS7960(r_en, l_en, r_pwm, l_pwm, freq)</code>	<code>gpio_num_t x4,</code> <code>uint8_t</code>	<code>void</code>	Constructs driver; configures LEDC timer and channels. Default frequency 20kHz is above audible range.
<code>setSpeed(speed, dir)</code>	<code>float 0.0-1.0,</code> <code>MOTOR_DIRECTION</code>	<code>void</code>	Sets normalized PWM duty cycle. <code>MOTOR_DIRECTION: FORWARD BACKWARD</code> . Internally maps to <code>r_pwm / l_pwm</code> activation.
<code>forward(speed)</code>	<code>float 0.0-1.0</code>	<code>void</code>	Convenience wrapper: sets direction FORWARD and applies speed duty cycle.
<code>backward(speed)</code>	<code>float 0.0-1.0</code>	<code>void</code>	Convenience wrapper: sets direction BACKWARD and applies speed duty cycle.
<code>stop()</code>	—	<code>void</code>	Sets both PWM channels to zero duty cycle. Motor coasts to a stop. Enable pins remain HIGH.

Method / Constructor	Parameters	Returns	Description
<code>brake()</code>	—	<code>void</code>	Activates both enable and PWM to maximum simultaneously, creating regenerative braking torque.
<code>enable()</code>	—	<code>void</code>	Asserts both <code>r_en</code> and <code>l_en</code> HIGH. Motor driver becomes active.
<code>disable()</code>	—	<code>void</code>	De-asserts both enable pins. Driver enters low-power tri-state mode. All PWM ceases.

2.1.4 Usage Example

```
// Instantiate driver on GPIO pins
BTS7960 motor(GPIO_NUM_25, GPIO_NUM_26, GPIO_NUM_27, GPIO_NUM_14);

motor.enable();           // Activate driver ICs
motor.forward(0.75f);     // 75% duty cycle forward
vTaskDelay(pdMS_TO_TICKS(2000));
motor.brake();            // Hard stop
motor.disable();          // Power down ICs
```

⚠ SAFETY NOTE

Never call `setSpeed()` with the driver disabled. Always call `enable()` first to prevent undefined behavior from floating enable pins.

HAL-02

BTS7960_Pot_Position — Closed-Loop Position Controller

2.2.1 Purpose & Hardware Context

This class transforms a BTS7960 motor into a high-torque servo actuator by combining the motor driver, a potentiometer position sensor, and a PID controller into a unified closed-loop system. It is architecturally a composite object: it owns (or references) instances of BTS7960, Potentiometer, and PID.

2.2.2 Constructor

```
BTS7960_Pot_Position(
    BTS7960& motor, // Reference to configured BTS7960 instance
    Potentiometer& pot, // Reference to configured Potentiometer instance
    float kp, // Proportional gain
```

```

float ki,           // Integral gain (anti-windup enforced)
float kd           // Derivative gain
);

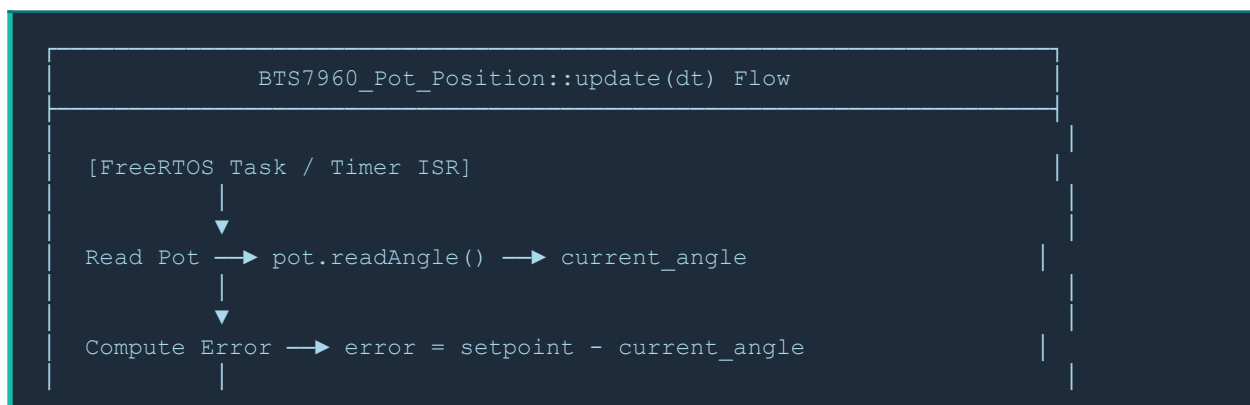
```

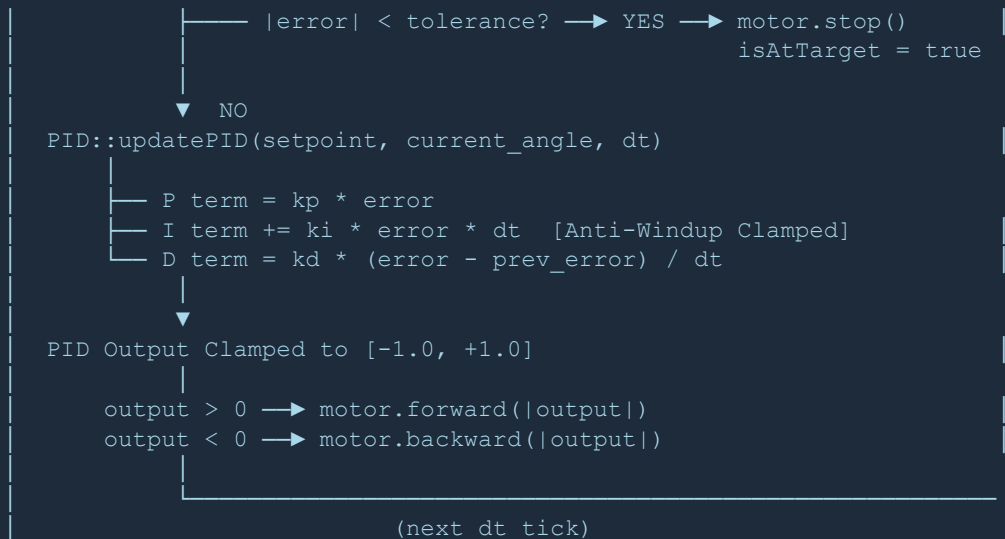
2.2.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>BTS7960_Pot_Position(motor, pot, kp, ki, kd)</code>	<code>BTS7960&</code> , <code>Potentiometer&</code> , <code>float x3</code>	—	Constructor. Initializes composite object. PID gains are forwarded to embedded PID instance. Motor is not enabled until <code>setAngle()</code> is first called.
<code>setAngle(target_deg)</code>	<code>float</code>	<code>void</code>	Commands target joint angle in degrees. Begins closed-loop PID control. Actuator remains powered until <code>isAtTarget()</code> returns true.
<code>getAngle()</code>	—	<code>float</code>	Reads current pot angle (degrees) via <code>Potentiometer::readAngle()</code> . Does not invoke PID computation.
<code>update(dt)</code>	<code>float</code> (delta-time, seconds)	<code>void</code>	Must be called at fixed intervals from a FreeRTOS task or timer ISR. Reads pot, computes PID output, drives motor speed+direction. Core control loop tick.
<code>isAtTarget()</code>	—	<code>bool</code>	Returns true when $ \text{current} - \text{setpoint} < \text{tolerance threshold}$ (configurable, default 2.0 degrees).

2.2.4 Position Control Loop (Mermaid Logic Flow)

The following represents the internal control loop executed on each call to `update(dt)`:





HAL-03 Potentiometer — Advanced ADC Driver

2.3.1 Purpose & Hardware Context

Provides a high-precision multi-modal ADC interface for rotary potentiometers. Implements ESP-IDF's `adc_cali_line_fitting` scheme for voltage compensation. All reads are optionally averaged to reject noise from motor EMI.

2.3.2 Constructor

```

Potentiometer(adc_channel_t channel, // ADC1 channel (e.g., ADC_CHANNEL_6)
               adc_atten_t atten,    // Attenuation (ADC_ATTEN_DB_11 for 0-3.3V)
               int max_angle = 270); // Full-scale physical rotation
(degrees)

```

2.3.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>readRaw()</code>	—	<code>int</code> (0-4095)	Returns raw 12-bit ADC conversion result without any processing.
<code>readVoltage()</code>	—	<code>float</code> (mV)	Returns calibrated voltage using <code>adc_cali_line_fitting</code> . More accurate than raw conversion.

Method / Constructor	Parameters	Returns	Description
<code>readAngle()</code>	—	<code>float</code> (deg)	Maps calibrated voltage to angular position. Range: 0 to <code>max_angle</code> degrees.
<code>readPercentage()</code>	—	<code>float</code> (0-100)	Expresses current position as percentage of full mechanical range.
<code>readMapped(min, max)</code>	<code>float, float</code>	<code>float</code>	Maps ADC reading to arbitrary user-defined numeric range [min, max].
<code>readAveraged(samples)</code>	<code>int</code> (1-64)	<code>float</code> (deg)	Performs N sequential reads, returns arithmetic mean in degrees. Reduces ADC noise and motor EMI spikes.
<code>setMaxAngle(angle)</code>	<code>int</code> (degrees)	<code>void</code>	Reconfigures full-scale angle for range calibration without reinstantiation.

i ADC CALIBRATION

The driver automatically performs `adc_cal_i_line_fitting` characterization on construction using eFuse-stored factory coefficients. If eFuse data is unavailable, the driver falls back to uncalibrated mode with a warning via `ESP_LOGW`.

HAL-04 MPU6050 — Inertial Measurement Unit**2.4.1 Purpose & Hardware Context**

Interfaces the InvenSense MPU-6050 6-DOF IMU over I2C. Exposes raw accelerometer and gyroscope data, and computes fused orientation angles using a complementary filter that balances gyroscopic drift correction against accelerometer noise.

2.4.2 Constructor

```
MPU6050(i2c_port_t port,          // I2C bus (I2C_NUM_0 or I2C_NUM_1)
        gpio_num_t sda,          // SDA GPIO
        gpio_num_t scl,          // SCL GPIO
        uint8_t   addr = 0x68,  // I2C address (0x68 AD0=LOW, 0x69 AD0=HIGH)
        uint32_t   freq = 400000); // I2C clock frequency (Hz)
```

2.4.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>readAccel(ax, ay, az)</code>	<code>float& x3</code> (output)	<code>esp_err_t</code>	Reads 3-axis acceleration. Values in g units ($\pm 2g$ default scale).
<code>readGyro(gx, gy, gz)</code>	<code>float& x3</code> (output)	<code>esp_err_t</code>	Reads 3-axis angular rate. Values in $^{\circ}/s$ ($\pm 250^{\circ}/s$ default scale).
<code>updateAngles(dt)</code>	<code>float</code> (seconds)	<code>void</code>	Computes fused Roll, Pitch, Yaw using complementary filter. Call at fixed dt from FreeRTOS timer. $\alpha = 0.96$ (gyro weight).
<code>getRoll()</code>	—	<code>float</code>	Returns current Roll angle in degrees (rotation about X-axis).
<code>getPitch()</code>	—	<code>float</code>	Returns current Pitch angle in degrees (rotation about Y-axis).
<code>getYaw()</code>	—	<code>float</code>	Returns current Yaw angle in degrees (rotation about Z-axis). Note: Yaw drifts without magnetometer.
<code>calibrate(samples)</code>	<code>int</code> (default 200)	<code>void</code>	Averages N readings at rest to compute accelerometer and gyroscope bias offsets. Call once at startup with device stationary.

2.4.4 Complementary Filter Mathematics

```
// Alpha = 0.96 blends 96% gyro (fast, drifts) with 4% accel (noisy, stable long-term)

accel_angle_x = atan2(ay, az) * RAD_TO_DEG;
accel_angle_y = atan2(-ax, sqrt(ay*ay + az*az)) * RAD_TO_DEG;

roll  = ALPHA * (roll  + gx * dt) + (1.0f - ALPHA) * accel_angle_x;
pitch = ALPHA * (pitch + gy * dt) + (1.0f - ALPHA) * accel_angle_y;
yaw   += gz * dt; // Gyro-only integration (no reference vector for correction)
```

HAL-05

Servo — Intelligent PWM Servo Controller

2.5.1 Purpose & Hardware Context

Provides unified control for both standard position servos ($0-180^{\circ}$) and continuous rotation servos (360°). Uses ESP32 LEDC peripheral to generate precise 50Hz PWM pulses ($500\mu s-2500\mu s$ range). Supports runtime mode switching.

2.5.2 Modes of Operation

Mode Constant	Behavior
STANDARD_180	Position servo. <code>setAngle(0-180)</code> maps linearly to 500µs-2500µs pulse width.
CONTINUOUS_360	Continuous rotation servo. <code>setSpeed()</code> with CLOCKWISE/COUNTERCLOCKWISE/STOP direction.

2.5.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>Servo(gpio, mode)</code>	<code>gpio_num_t</code> , <code>ServoMode</code>	—	Constructor. Initializes LEDC channel at 50Hz. Mode determines behavior of subsequent control calls.
<code>attach()</code>	—	<code>void</code>	Begins PWM signal output. Servo becomes powered and responsive.
<code>detach()</code>	—	<code>void</code>	Stops PWM output. Servo de-energizes (position may drift under load).
<code>setAngle(angle)</code>	<code>float</code> (0-180)	<code>void</code>	STANDARD_180 mode only. Maps angle to pulse width. Clamped to [0, 180].
<code>setSpeed(speed, dir)</code>	<code>float</code> 0-100, <code>ROTATION_DIRECTION</code>	<code>void</code>	CONTINUOUS_360 mode only. Speed 0 = stop. Direction: CW or CCW. Maps to deviation from 1500µs center.
<code>setPulse(us)</code>	<code>uint32_t</code> (500-2500)	<code>void</code>	Direct pulse width control in microseconds. Advanced use; bypasses mode logic.
<code>getAngle()</code>	—	<code>float</code>	Returns last commanded angle (STANDARD_180 mode). Not a physical readback.

```
// Standard 180-degree servo
Servo arm(GPIO_NUM_18, STANDARD_180);
arm.attach();
arm.setAngle(90.0f); // Center position

// Continuous rotation servo
Servo wheel(GPIO_NUM_19, CONTINUOUS_360);
wheel.attach();
wheel.setSpeed(50.0f, CLOCKWISE); // 50% speed clockwise
```

HAL-06

PID — Proportional-Integral-Derivative Controller

2.6.1 Purpose & Design Philosophy

A standalone, reusable PID controller class with industrial-grade features including integral anti-windup and output clamping. Designed to be embedded within higher-level controllers (e.g., `BTS7960_Pot_Position`) or used directly in application code.

2.6.2 Constructor

```
PID(float kp,           // Proportional gain
    float ki,           // Integral gain
    float kd,           // Derivative gain
    float output_min = -1.0f, // Output clamp lower bound
    float output_max = 1.0f); // Output clamp upper bound
```

2.6.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>updatePID(setpoint, measurement, dt)</code>	<code>float, float, float</code>	<code>float</code>	Core computation. Returns clamped PID output. Must be called at consistent dt intervals for accurate derivative and integral terms.
<code>reset()</code>	—	<code>void</code>	Zeroes integral accumulator and previous error. Call when setpoint changes significantly to prevent integral kick.
<code>setGains(kp, ki, kd)</code>	<code>float x3</code>	<code>void</code>	Runtime gain tuning without reinstantiation. Useful for adaptive control or commissioning.
<code>setOutputLimits(min, max)</code>	<code>float, float</code>	<code>void</code>	Reconfigures output clamping range. Anti-windup integrator clamp is also updated to match.
<code>getProportional()</code>	—	<code>float</code>	Returns last computed P-term (diagnostic).
<code>getIntegral()</code>	—	<code>float</code>	Returns current I-term accumulator (diagnostic).
<code>getDerivative()</code>	—	<code>float</code>	Returns last computed D-term (diagnostic).

2.6.4 Anti-Windup Implementation

Integrator windup occurs when the output is saturated yet the integral continues accumulating. This causes overshoot and instability when the system re-enters controllable range. The implementation prevents this by clamping the integral accumulator:

```
float PID::updatePID(float setpoint, float measurement, float dt) {
    float error = setpoint - measurement;

    // Proportional term
    float p_term = kp * error;

    // Integral term with Anti-Windup clamp
    integral += ki * error * dt;
    integral = std::clamp(integral, output_min, output_max); // <-- Anti-Windup

    // Derivative term (on measurement to prevent derivative kick)
    float d_term = kd * (error - prev_error) / dt;
    prev_error = error;

    // Sum and clamp output
    float output = p_term + integral + d_term;
    return std::clamp(output, output_min, output_max); // <-- Output Clamping
}
```

HAL-07

BTS7960_Position — Encoder-Based Position Control (Legacy)

2.7.1 Overview

A legacy position controller that uses a quadrature encoder instead of a potentiometer for position feedback. Relies on a hardware interrupt service routine (ISR) to count encoder pulses at high frequency without software polling overhead.

2.7.2 ISR Architecture

```
// ISR registered on encoder channel A rising/falling edge
static void IRAM_ATTR encoder_isr_handler(void* arg) {
    BTS7960_Position* self = (BTS7960_Position*)arg;
    // Read channel B to determine direction
    if (gpio_get_level(self->enc_b_pin)) {
        self->pulse_count++;
    } else {
        self->pulse_count--;
    }
}

// Registration during construction
gpio_isr_handler_add(enc_a_pin, encoder_isr_handler, this);
```

⚠ IRAM REQUIREMENT

The `encoder_isr_handler` function is decorated with `IRAM_ATTR`. This places it in Internal RAM, ensuring it executes even when the instruction cache is busy. Failure to use `IRAM_ATTR` will cause ISR panics on cache misses.

2.7.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>BTS7960_Position(motor, enc_a, enc_b, ppr)</code>	<code>BTS7960&</code> , <code>gpio_num_t x2</code> , <code>int</code>	—	Constructor. Registers encoder ISR. <code>ppr</code> = pulses per revolution of encoder.
<code>setPosition(pulses)</code>	<code>int32_t</code>	<code>void</code>	Commands target encoder count. Begins PID-driven positioning.
<code>getPosition()</code>	—	<code>int32_t</code>	Returns current pulse count from ISR accumulator.
<code>resetEncoder()</code>	—	<code>void</code>	Atomically zeroes <code>pulse_count</code> . Use at known reference position.
<code>update(dt)</code>	<code>float</code>	<code>void</code>	Runs PID loop. Must be called from task at fixed <code>dt</code> .

HAL-08**tcrt_sensor — TCRT5000 Reflective Line Sensor****2.8.1 Overview**

Interfaces the TCRT5000 IR reflectance sensor in both digital threshold mode and analog raw mode. Typically deployed in arrays for line-following applications.

Method / Constructor	Parameters	Returns	Description
<code>tcrt_sensor(digital_pin, adc_channel)</code>	<code>gpio_num_t</code> , <code>adc_channel_t</code>	—	Constructor. Configures GPIO for digital read and ADC channel for analog read.
<code>readDigital()</code>	—	<code>int</code> (0 or 1)	Returns HIGH if reflectance exceeds hardware comparator threshold (white surface), LOW for dark surface.
<code>readAnaloge()</code>	—	<code>int</code> (0-4095)	Returns raw ADC value proportional to IR reflectance intensity. Useful for gradient detection.

HAL-09 ultrasonic — HC-SR04 Ultrasonic Distance Sensor**2.9.1 Overview**

Drives the HC-SR04 sensor via a 10µs trigger pulse and measures echo pulse width using `esp_timer_get_time()` for µs precision. Distance is derived from the speed of sound in air at standard conditions.

```
// Measurement equation:
// distance_cm = echo_pulse_us / 58.0f
// (based on sound @ 343 m/s: round trip = 2 * distance)

float ultrasonic::readDistance() {
    // Assert TRIG for 10µs
    gpio_set_level(trig_pin, 1);
    esp_rom_delay_us(10);
    gpio_set_level(trig_pin, 0);

    // Wait for ECHO to go HIGH (start of pulse)
    int64_t start = esp_timer_get_time();
    while (!gpio_get_level(echo_pin) && (esp_timer_get_time() - start) < 30000);

    // Measure HIGH duration
    int64_t pulse_start = esp_timer_get_time();
    while (gpio_get_level(echo_pin));
    int64_t pulse_end = esp_timer_get_time();

    return (float)(pulse_end - pulse_start) / 58.0f;
}
```

Method / Constructor	Parameters	Returns	Description
<code>ultrasonic(trig, echo)</code>	<code>gpio_num_t x2</code>	—	Constructor. Configures trig as OUTPUT, echo as INPUT.
<code>readDistance()</code>	—	<code>float (cm)</code>	Triggers measurement and returns distance in centimeters. Blocks up to 30ms on echo timeout. Returns -1.0f on timeout.

HAL-10 dc_motor — Generic Vector-Based DC Motor Driver**2.10.1 Overview**

A lightweight driver supporting multi-pin motor bridges. Abstracts FORWARD, BACKWARD, LEFT, RIGHT directional vectors for holonomic or differential drive configurations.

Method / Constructor	Parameters	Returns	Description
<code>dc_motor(pin_config[])</code>	<code>dc_motor_pins_t</code> array	—	Constructor. Accepts flexible pin array for L298N, TB6612, or custom H-bridge wiring.
<code>dc_move(DIRECTION)</code>	<code>DIRECTION</code> enum	<code>void</code>	Executes movement vector: FORWARD, BACKWARD, LEFT, RIGHT, STOP. Maps to GPIO assertion patterns.

HAL-11 Button — Debounced Digital Input

Method / Constructor	Parameters	Returns	Description
<code>Button(pin, mode)</code>	<code>gpio_num_t</code> , <code>GPIO_MODE</code> (PULLUP/PULLDOWN)	—	Constructor. Configures GPIO with internal pull resistor.
<code>GET()</code>	—	<code>int</code> (0 or 1)	Returns debounced button state. Inverts logic if PULLUP mode (pressed = 0 becomes logical 1).

i DEBOUNCE

Software debouncing uses a 20ms lockout window implemented via `esp_timer`. Hardware RC debounce is recommended for critical inputs in high-EMI environments.

HAL-12 Led — Digital Output with Timed Toggle

Method / Constructor	Parameters	Returns	Description
<code>Led(pin)</code>	<code>gpio_num_t</code>	—	Constructor. Configures GPIO as push-pull output.
<code>ON()</code>	—	<code>void</code>	Sets GPIO HIGH. LED illuminates.
<code>OFF()</code>	—	<code>void</code>	Sets GPIO LOW. LED extinguishes.
<code>TOGGLE(period_ms)</code>	<code>uint32_t</code>	<code>void</code>	Continuously toggles LED at $\text{period_ms}/2$ on, $\text{period_ms}/2$ off. Uses FreeRTOS <code>vTaskDelay</code> internally.

3. Microcontroller Abstraction Layer (MCAL)

The MCAL provides silicon-level abstractions for connectivity, persistence, and serial communication. It decouples application and HAL code from ESP-IDF API specifics, enabling portability and unit testability.

MCAL-0
1

WiFiManager — Station Mode Connection Manager

3.1.1 Overview

Manages the full WiFi Station lifecycle: initialization, scanning, connection, IP acquisition, reconnection backoff, and event dispatching. Built on ESP-IDF's event loop mechanism.

3.1.2 Constructor

```
WiFiManager(const char* ssid,           // Network SSID
             const char* password,      // WPA2 passphrase
             uint8_t max_retries = 5);  // Reconnect attempts before giving up
```

3.1.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>WiFiManager(ssid, password, retries)</code>	<code>const char* x2,</code> <code>uint8_t</code>	—	Constructor. Registers WIFI_EVENT and IP_EVENT handlers with ESP-IDF event loop.
<code>connect()</code>	—	<code>esp_err_t</code>	Initiates WiFi connection. Non-blocking; completion signalled via <code>on_connected</code> callback.
<code>disconnect()</code>	—	<code>void</code>	Gracefully disconnects from AP. Triggers <code>on_disconnected</code> callback.
<code>isConnected()</code>	—	<code>bool</code>	Returns current connection state from internal flag (no network round-trip).
<code>on_connected(ip)</code>	<code>std::string</code> (IP addr)	<code>callback</code>	User-provided callback invoked when IP address is successfully assigned. Suitable for MQTT connect trigger.
<code>on_disconnected()</code>	—	<code>callback</code>	User-provided callback invoked on disconnect. Allows higher layers to pause MQTT operations.

Method / Constructor	Parameters	Returns	Description
<code>getIP()</code>	—	<code>std::string</code>	Returns last assigned IPv4 address as dotted-decimal string.

```

WiFiManager wifi("MyNetwork", "MyPassword");

wifi.on_connected([](std::string ip) {
    ESP_LOGI(TAG, "Connected! IP: %s", ip.c_str());
    mqtt.connect(); // Start MQTT after WiFi is up
});

wifi.on_disconnected([]() {
    ESP_LOGW(TAG, "WiFi lost - pausing MQTT");
});

wifi.connect();

```

MCAL-0 2

MQTTClient — MQTT 3.1.1 Pub/Sub Wrapper

3.2.1 Overview

Wraps `esp_mqtt_client` with a clean C++ pub/sub interface. Supports per-topic callbacks via a topic-to-handler routing table. QoS 0 and QoS 1 are supported.

3.2.2 API Reference

Method / Constructor	Parameters	Returns	Description
<code>MQTTClient(broker_url)</code>	<code>const char*</code>	—	Constructor. Configures <code>esp_mqtt_client_config_t</code> . Does not connect until <code>connect()</code> is called.
<code>connect()</code>	—	<code>esp_err_t</code>	Initiates TCP connection to MQTT broker. Connection success signalled via internal event handler.
<code>publish(topic, data, qos)</code>	<code>const char*</code> , <code>const char*</code> , <code>int</code>	<code>int</code>	Publishes message. Returns message ID (QoS 1) or 0 (QoS 0). Non-blocking.
<code>subscribe(topic, qos)</code>	<code>const char*</code> , <code>int</code> (0 or 1)	<code>esp_err_t</code>	Subscribes to topic pattern. Wildcards + and # supported.
<code>on_message(callback)</code>	<code>function(topic, data)</code>	<code>void</code>	Registers global message handler called for all incoming messages.

Method / Constructor	Parameters	Returns	Description
<code>on_topic(topic, callback)</code>	<code>const char*</code> , <code>function(data)</code>	<code>void</code>	Registers per-topic callback. Overrides global handler for matched topics. String matching only (no wildcard routing).
<code>disconnect()</code>	—	<code>void</code>	Gracefully sends MQTT DISCONNECT packet and closes TCP connection.

```

MQTTClient mqtt("mqtt://192.168.1.100:1883");

// Per-topic callback routing
mqtt.on_topic("/sarcoos/motor/1", [](std::string data) {
    float speed = std::stof(data);
    motor1.setSpeed(speed, FORWARD);
});

mqtt.connect();
mqtt.subscribe("/sarcoos/#"); // Subscribe to all Sarcoos topics

// Publishing from Suit side
mqtt.publish("/sarcoos/pot/1", std::to_string(pot1.readAngle()).c_str());

```

MCAL-0 3

NVS — Template-Based Non-Volatile Storage

3.3.1 Overview

A type-safe C++ wrapper over ESP-IDF's `nvs_flash` API. Uses templates to provide compile-time type safety for persisted configuration values such as WiFi credentials, PID gains, and calibration data.

3.3.2 API Reference

Method / Constructor	Parameters	Returns	Description
<code>NVS(namespace)</code>	<code>const char*</code>	—	Constructor. Opens or creates NVS namespace. Initializes <code>nvs_flash</code> if required.
<code>write<T>(key, value)</code>	<code>const char*</code> , <code>T</code>	<code>esp_err_t</code>	Persists value under key. T: <code>int8/16/32/64</code> , <code>uint8/16/32/64</code> , <code>float</code> (stored as blob), <code>std::string</code> .
<code>read<T>(key, default_val)</code>	<code>const char*</code> , <code>T</code>	<code>T</code>	Reads persisted value. Returns <code>default_val</code> if key not found or type mismatch.

Method / Constructor	Parameters	Returns	Description
<code>erase(key)</code>	<code>const char*</code>	<code>esp_err_t</code>	Removes single key from namespace.
<code>eraseAll()</code>	—	<code>esp_err_t</code>	Erases all keys in namespace. Irreversible.
<code>commit()</code>	—	<code>esp_err_t</code>	Flushes pending writes to flash. Call after write operations to guarantee persistence.

```
NVS config("sarcoos_cfg");

// Persist PID gains
config.write<float>("kp_motor1", 2.5f);
config.write<float>("ki_motor1", 0.1f);
config.write<float>("kd_motor1", 0.05f);
config.commit();

// Read back with defaults
float kp = config.read<float>("kp_motor1", 1.0f); // Returns 2.5f
float kx = config.read<float>("kp_motor9", 1.0f); // Returns 1.0f (default)
```

MCAL-0 4

uart — Device-Structured UART Communication

3.4.1 Overview

Abstracts ESP-IDF UART driver with a device descriptor struct pattern. Enables clean initialization from a single config struct, analogous to I2C and SPI device tree patterns in Linux.

3.4.2 `uart_device_t` Configuration Struct

```
typedef struct {
    uart_port_t    port;           // UART_NUM_0, UART_NUM_1, UART_NUM_2
    int            baud_rate;      // e.g., 115200, 9600, 57600
    uart_parity_t  parity;        // UART_PARITY_DISABLE, EVEN, ODD
    uart_stop_bits_t stop_bits;   // UART_STOP_BITS_1 or _2
    gpio_num_t     tx_pin;        // TX GPIO
    gpio_num_t     rx_pin;        // RX GPIO
    size_t         rx_buf_size;   // RX ring buffer size (bytes)
} uart_device_t;
```

3.4.3 API Reference

Method / Constructor	Parameters	Returns	Description
<code>uart(uart_device_t config)</code>	<code>uart_device_t</code>	—	Constructor. Installs UART driver with specified RX buffer. Configures hardware flow control disabled by default.
<code>write(data, len)</code>	<code>const uint8_t*, size_t</code>	<code>int</code>	Transmits len bytes. Returns number of bytes written. Blocking until FIFO accepts data.
<code>read(buf, len, timeout_ms)</code>	<code>uint8_t*, size_t, int</code>	<code>int</code>	Reads up to len bytes. Returns actual bytes read. <code>timeout_ms</code> : -1 blocks indefinitely.
<code>flush()</code>	—	<code>void</code>	Clears RX ring buffer and hardware FIFO.
<code>available()</code>	—	<code>size_t</code>	Returns number of bytes waiting in RX buffer without blocking.

4. Application Layer Architecture

The Application Layer contains the mission-specific logic that orchestrates HAL and MCAL components to implement the Suit input pipeline and the Rover output pipeline. All application files run under FreeRTOS and are time-scheduled.

4.1 main.cpp — System Entry Point

The `app_main()` function serves as the ESP-IDF application entry point, equivalent to `main()` in standard C. It is responsible for the strictly ordered initialization sequence before spawning FreeRTOS tasks.

4.1.1 Initialization Sequence

```
extern "C" void app_main(void) {
    // 1. Initialize NVS flash partition
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND)
    {
        nvs_flash_erase();
        nvs_flash_init();
    }

    // 2. Initialize event loop (required for WiFi/MQTT events)
    esp_event_loop_create_default();

    // 3. Construct WiFiManager and MQTTClient (MCAL)
    WiFiManager wifi(SSID, PASSWORD);
    MQTTClient mqtt(BROKER_URL);

    // 4. Register WiFi callbacks — MQTT connect on IP acquired
    wifi.on_connected([&mqtt](std::string ip) { mqtt.connect(); });

    // 5. Start WiFi (non-blocking)
    wifi.connect();

    // 6. Spawn application tasks
    xTaskCreate(input_task, "inputs", 4096, &mqtt, 5, NULL); // Suit
    xTaskCreate(output_task, "outputs", 4096, &mqtt, 5, NULL); // Rover

    // app_main returns — FreeRTOS scheduler takes over
}
```

i TASK PRIORITY

Both `input_task` and `output_task` are assigned priority 5 (mid-range). The FreeRTOS idle task runs at priority 0. For time-critical PID update tasks, consider priority 7-9 to prevent jitter from lower-priority tasks preempting the control loop.

4.2 esp32_inputs.cpp — Suit Input Pipeline

Implements the `input_task` FreeRTOS task. This task runs on the Suit ESP32 and is responsible for polling all sensors, encoding their values, and publishing them to MQTT topics.

4.2.1 Input Resources

Resource	MQTT Topic Published
Potentiometer 1-10 (ADC)	/sarcoos/pot/1 through /sarcoos/pot/10
Button 1-4 (GPIO)	/sarcoos/btn/1 through /sarcoos/btn/4
MPU6050 Roll	/sarcoos/imu/roll
MPU6050 Pitch	/sarcoos/imu/pitch
MPU6050 Yaw	/sarcoos/imu/yaw
TCRT5000 Line Sensors	/sarcoos/line/1 through /sarcoos/line/4
HC-SR04 Distance	/sarcoos/dist/front

4.2.2 Task Implementation Pattern

```
static void input_task(void* arg) {
    MQTTClient* mqtt = (MQTTClient*)arg;

    // Initialize all HAL drivers
    Potentiometer pots[10] = { /* ADC channel configs */ };
    Button buttons[4] = { /* GPIO configs */ };
    MPU6050 imu(I2C_NUM_0, SDA_PIN, SCL_PIN);

    imu.calibrate(200); // Calibrate at startup

    TickType_t last_wake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(20); // 50Hz sample rate

    while (true) {
        // Read and publish potentiometers
        for (int i = 0; i < 10; i++) {
            float angle = pots[i].readAveraged(4);
            char topic[32], data[16];
            snprintf(topic, sizeof(topic), "/sarcoos/pot/%d", i+1);
            snprintf(data, sizeof(data), "%.2f", angle);
            mqtt->publish(topic, data);
        }

        // Read and publish IMU
        imu.updateAngles(0.020f); // dt = 20ms
        char roll_str[16];
        snprintf(roll_str, sizeof(roll_str), "%.2f", imu.getRoll());
        mqtt->publish("/sarcoos/imu/roll", roll_str);
    }
}
```

```

        vTaskDelayUntil(&last_wake, period); // Precise 50Hz pacing
    }
}

```

i TIMING DISCIPLINE

`vTaskDelayUntil()` is used instead of `vTaskDelay()`. This compensates for variable execution time within the task body, maintaining a precise 20ms period regardless of MQTT publish latency.

4.3 esp32_outputs.cpp — Rover Output Pipeline

Implements the `output_task` FreeRTOS task running on the Rover ESP32. This task subscribes to MQTT topics and actuates motors in response.

4.3.1 Subscription Map

MQTT Topic Subscribed	Actuator Action
<code>/sarcoos/pot/1</code>	BTS7960 Motor 1 speed and direction
<code>/sarcoos/pot/2</code>	BTS7960 Motor 2 speed and direction
<code>/sarcoos/pot/3-6</code>	Servo arm positions (degrees)
<code>/sarcoos/pot/7-10</code>	BTS7960_Pot_Position target angles
<code>/sarcoos/btn/1</code>	Emergency stop — all motors
<code>/sarcoos/btn/2</code>	Mode switch (open-loop / closed-loop)
<code>/sarcoos/imu/roll</code>	Stabilization compensation input

4.3.2 Task Implementation Pattern

```

static void output_task(void* arg) {
    MQTTClient* mqtt = (MQTTClient*)arg;

    // Initialize motors
    BTS7960 motor1(R_EN1, L_EN1, R_PWM1, L_PWM1);
    motor1.enable();

    // Register per-topic callbacks (non-blocking event-driven)
    mqtt->on_topic("/sarcoos/pot/1", [&motor1](std::string data) {
        float val = std::stof(data);
        // Map 0-270 deg pot range to -1.0 to +1.0 motor speed
        float speed = (val / 135.0f) - 1.0f;
        if (speed > 0) motor1.forward(speed);
        else if (speed < 0) motor1.backward(-speed);
        else motor1.stop();
    });
}

```



```

});

mqtt->on_topic("/sarcoos/btn/1", [&motor1](std::string data) {
    if (data == "1") motor1.brake(); // Emergency stop
});

// Subscribe to all topics
mqtt->subscribe("/sarcoos/#");

// Task becomes idle - callbacks handle all actuation
while (true) { vTaskDelay(pdMS_TO_TICKS(100)); }
}

```

4.4 pot_pos_control.cpp — High-Torque Servo Implementation

Demonstrates the practical implementation of `BTS7960_Pot_Position` as a high-torque servo replacement. This pattern is used for heavy joint actuation where standard RC servos lack sufficient torque.

```

// Hardware setup
BTS7960      motor(R_EN, L_EN, R_PWM, L_PWM);
Potentiometer pot(ADC_CHANNEL_6, ADC_ATTEN_DB_11, 270);
BTS7960_Pot_Position joint(motor, pot, KP, KI, KD);

// PID gains (tune per mechanical assembly)
#define KP  3.5f // Aggressive proportional for heavy joint
#define KI  0.05f // Small integral to eliminate steady-state
#define KD  0.2f // Derivative for damping oscillation

// Control task - 50Hz update rate
static void pos_control_task(void* arg) {
    BTS7960_Pot_Position* joint = (BTS7960_Pot_Position*)arg;
    TickType_t last = xTaskGetTickCount();

    while (true) {
        joint->update(0.020f); // 20ms dt

        if (joint->isAtTarget()) {
            // Optional: reduce motor duty to holding current
        }

        vTaskDelayUntil(&last, pdMS_TO_TICKS(20));
    }
}

// Command from MQTT callback
mqtt.on_topic("/sarcoos/pot/7", [&joint](std::string data) {
    float target_deg = std::stof(data); // 0-270 degrees
    joint.setAngle(target_deg);
    joint.reset(); // Reset PID integral on new target
}

```

```
});
```

5. Performance & Safety Features

5.1 FreeRTOS Threading Model

The system uses dedicated FreeRTOS tasks with explicit stack sizes and priorities to ensure deterministic timing and prevent stack overflow failures.

Task Name	Description
<code>input_task (Suit)</code>	50Hz polling of all sensors. Priority 5, stack 4096B.
<code>output_task (Rover)</code>	Event-driven motor actuation. Priority 5, stack 4096B.
<code>pos_control_task</code>	50Hz PID position loop. Priority 7 (higher for timing). Stack 2048B.
<code>MQTT event loop</code>	Managed internally by <code>esp_mqtt_client</code> . Runs as separate task.
<code>WiFi event loop</code>	Managed by ESP-IDF <code>netif</code> . Handles DHCP and reconnect.

5.2 PID Safety Architecture

Two safety mechanisms prevent instability:

Integral Anti-Windup:

When the motor output is physically limited (e.g., mechanically blocked), the PID integral would normally accumulate unboundedly. The anti-windup mechanism clamps the integral accumulator to `[output_min, output_max]`, preventing large overshoots when the obstruction is removed.

Output Clamping:

The total PID output (P+I+D) is clamped to `[-1.0, +1.0]` before being applied to `BTS7960::setSpeed()`. This prevents requests for duty cycles exceeding hardware limits and protects against current spikes.

5.3 ADC Calibration Scheme

Raw ADC readings from the ESP32 have significant non-linearity due to the SAR ADC architecture. The Potentiometer driver uses ESP-IDF's `adc_cali_line_fitting` scheme, which applies a third-order polynomial correction derived from factory-stored eFuse calibration coefficients. This reduces voltage reading error from $\pm 150\text{mV}$ (raw) to $\pm 30\text{mV}$ (calibrated).

```
// Calibration initialization (Potentiometer constructor)
adc_cali_line_fitting_config_t cali_config = {
```

```

        .unit_id = ADC_UNIT_1,
        .atten   = atten,
        .bitwidth = ADC_BITWIDTH_12,
        .default_vref = 1100 // mV fallback if eFuse unavailable
    };
    adc_cali_create_scheme_line_fitting(&cali_config, &cali_handle);

```

5.4 MQTT Reliability Considerations

The system operates at QoS 0 (fire-and-forget) for real-time telemetry (potentiometer values) and QoS 1 (at-least-once delivery) for critical commands (emergency stop buttons). This balances latency with reliability.

Scenario	Mitigation
WiFi dropout	WiFiManager reconnects automatically with exponential backoff up to max_retries.
MQTT broker disconnect	MQTTClient uses clean_session=false with QoS 1 for emergency commands to ensure delivery on reconnect.
Motor stall / overcurrent	BTS7960 disable() de-energizes driver ICs. Application watchdog should call disable() on repeated stall detection.
PID windup on joint block	Anti-windup clamp prevents integral explosion. Motor continues to receive bounded output.
ADC noise from motor EMI	readAveraged(N) smooths N samples. Increasing N trades latency for noise rejection.

5.5 Memory Budget

Component	Stack / Heap Allocation
input_task	4096 bytes stack
output_task	4096 bytes stack
pos_control_task	2048 bytes stack
MQTTClient internal	~8KB heap (rx/tx buffers + esp_mqtt_client)
WiFiManager	~16KB heap (TCP/IP stack via lwIP)
NVS partition	16KB flash (default nvs partition)
Total heap budget (typical)	~40KB of ~320KB available (safe margin)

6. Quick Reference — Complete API Index

6.1 HAL Drivers Summary

Class	Key Methods	Primary Use Case
<code>BTS7960</code>	<code>forward</code> , <code>backward</code> , <code>brake</code> , <code>stop</code> , <code>enable</code> , <code>disable</code>	High-power DC motor open-loop speed control
<code>BTS7960_Pot_Position</code>	<code>setAngle</code> , <code>update</code> , <code>isAtTarget</code> , <code>getAngle</code>	Closed-loop joint positioning (servo replacement)
<code>BTS7960_Position</code>	<code>setPosition</code> , <code>update</code> , <code>resetEncoder</code>	Encoder-based position control (legacy)
<code>Potentiometer</code>	<code>readAngle</code> , <code>readAveraged</code> , <code>readMapped</code>	Angular position sensing with noise filtering
<code>MPU6050</code>	<code>readAccel</code> , <code>readGyro</code> , <code>updateAngles</code>	6-DOF inertial sensing for orientation
<code>Servo</code>	<code>setAngle</code> , <code>setSpeed</code> , <code>attach</code> , <code>detach</code>	RC servo position and continuous rotation
<code>PID</code>	<code>updatePID</code> , <code>reset</code> , <code>setGains</code>	Reusable PID controller with anti-windup
<code>tqrt_sensor</code>	<code>readDigital</code> , <code>readAnalog</code>	IR reflectance for line following
<code>ultrasonic</code>	<code>readDistance</code>	HC-SR04 obstacle detection
<code>dc_motor</code>	<code>dc_move(DIRECTION)</code>	Simple directional motor control
<code>Button</code>	<code>GET()</code>	Debounced digital input reading
<code>Led</code>	<code>ON</code> , <code>OFF</code> , <code>TOGGLE</code>	GPIO output with timed toggling

6.2 MCAL Components Summary

Class	Key Methods	Primary Use Case
<code>WiFiManager</code>	<code>connect</code> , <code>disconnect</code> , <code>on_connected</code> , <code>on_disconnected</code>	WiFi station lifecycle with event callbacks
<code>MQTTClient</code>	<code>publish</code> , <code>subscribe</code> , <code>on_message</code> , <code>on_topic</code>	MQTT pub/sub with per-topic callback routing

Class	Key Methods	Primary Use Case
<code>NVS</code>	<code>write<T></code> , <code>read<T></code> , <code>commit</code> , <code>erase</code>	Type-safe persistent key-value storage in flash
<code>uart</code>	<code>write</code> , <code>read</code> , <code>available</code> , <code>flush</code>	Device-structured serial communication

6.3 MQTT Topic Registry

Complete listing of all MQTT topics used in the Sarcoos ecosystem:

Topic Pattern	Direction	QoS	Payload / Description
<code>/sarcoos/pot/1-10</code>	Suit → Rover	QoS 0	float (degrees 0.00-270.00). Potentiometer angle.
<code>/sarcoos/btn/1-4</code>	Suit → Rover	QoS 1	int (0 or 1). Button state. QoS 1 for critical commands.
<code>/sarcoos/imu/roll</code>	Suit → Rover	QoS 0	float (degrees). Roll angle from MPU6050.
<code>/sarcoos/imu/pitch</code>	Suit → Rover	QoS 0	float (degrees). Pitch angle.
<code>/sarcoos/imu/yaw</code>	Suit → Rover	QoS 0	float (degrees). Yaw angle (gyro-only, drifts).
<code>/sarcoos/line/1-4</code>	Suit → Rover	QoS 0	int (0 or 1). TCRT5000 line sensor state.
<code>/sarcoos/dist/front</code>	Suit → Rover	QoS 0	float (cm). HC-SR04 distance.
<code>/sarcoos/status/suit</code>	Suit → Broker	QoS 1	string (online/offline). LWT last will topic.
<code>/sarcoos/status/rover</code>	Rover → Broker	QoS 1	string (online/offline). LWT last will topic.

6.4 GPIO Pin Assignment Reference

Recommended GPIO assignments for the standard Sarcoos v1 hardware configuration:

Signal	GPIO / Notes
BTS7960 #1 R_EN	GPIO_NUM_25
BTS7960 #1 L_EN	GPIO_NUM_26
BTS7960 #1 R_PWM	GPIO_NUM_27 (LEDC Channel 0)
BTS7960 #1 L_PWM	GPIO_NUM_14 (LEDC Channel 1)
Potentiometer 1-6 (ADC)	ADC1_CH0 through ADC1_CH5 (GPIO36-GPIO39, GPIO32-GPIO33)
MPU6050 SDA	GPIO_NUM_21 (I2C_NUM_0 default)
MPU6050 SCL	GPIO_NUM_22 (I2C_NUM_0 default)

Signal	GPIO / Notes
Servo 1-4	GPIO_NUM_18/19/23/5 (LEDC Channels 2-5)
HC-SR04 TRIG	GPIO_NUM_12
HC-SR04 ECHO	GPIO_NUM_13
TCRT5000 Digital	GPIO_NUM_34-GPIO_NUM_35 (input-only)
Encoder A/B	GPIO_NUM_4/GPIO_NUM_16 (interrupt capable)